

AI Mini Project

Lane Detection using Classical Computer Vision

Provisional / Draft Project Report

Item	Details
Project Title	Lane Detection (Project No. 33)
Team Members	[Student 1 Name – Roll No.] [Student 2 Name – Roll No.] [Student 3 Name – Roll No.]
Course / Review	AI Mini Project – Draft Review
Submission	Draft (work in progress)

1. Problem Statement & Objectives

Problem statement

Lane detection is the task of locating the left and right lane markings of the road ahead from a front-facing camera image. It is a core building block of driver-assistance systems (lane departure warning, lane keeping) and autonomous driving. Lane markings vary in colour (white, yellow), shape (solid, dashed, curved) and visibility (shadows, glare, worn paint), which makes a simple, robust detector non-trivial.

Objectives

- Build a working lane detection pipeline using OpenCV that marks the left and right lane boundaries on road images and video.
- Analyse the dataset (image size, brightness, contrast, scene variety) to understand the conditions the detector must handle.
- Make video output stable using temporal smoothing and measure the improvement.
- Measure and report results (detection rate, stability, speed) and document limitations honestly.
- (Remaining) Evaluate against labelled ground truth on TuSimple and compare with a deep learning model.

2. Dataset & Source

The assigned reference dataset for this project is TuSimple (<https://www.kaggle.com/datasets/manideep1108/tusimple>), which provides 1280×720 highway clips with labelled lane coordinates. For this draft, the working dataset is the Udacity CarND Lane Lines sample set, which is publicly available on GitHub and was downloaded directly:

- Source: <https://github.com/udacity/CarND-LaneLines-P1> (folders test_images and test_videos)
- Contents: 6 still images (960×540) and 3 driving videos (960×540 and 1280×720), totalling 1,153 video frames.
- Labels: none. This set has no ground-truth lane annotations, so accuracy against labels cannot be computed on it. This is why TuSimple is planned for the final evaluation.

Note: TuSimple could not be downloaded in the environment used to prepare this draft. Switching to it is the first item in remaining work.

3. Basic Data Analysis

Sample	Type	Resolution	Frames	Mean brightness	Contrast (std)
solidWhiteCurve.jpg	image	960×540	1	131.2	41.7
solidWhiteRight.jpg	image	960×540	1	130.1	43.8
solidYellowCurve.jpg	image	960×540	1	129.1	34.0
solidYellowCurve2.jpg	image	960×540	1	134.7	33.3
solidYellowLeft.jpg	image	960×540	1	130.4	35.0
whiteCarLaneSwitch.jpg	image	960×540	1	135.5	31.9

Sample	Type	Resolution	Frames	Mean brightness	Contrast (std)
solidWhiteRight.mp4	video	960×540	221	124.7	44.4
solidYellowLeft.mp4	video	960×540	681	128.3	34.5
challenge.mp4	video	1280×720	251	112.2	47.7

Observations: the still images and the two simple videos are bright daylight scenes with similar brightness (about 124–136). The challenge video is the darkest (112.2) and has the highest contrast (47.7) because of tree shadows across the road and a change of road surface colour, which makes it the hardest case. The set covers white and yellow markings, solid and dashed lines, gentle curves and a lane-change scene, but contains no night, rain or heavily occluded scenes.

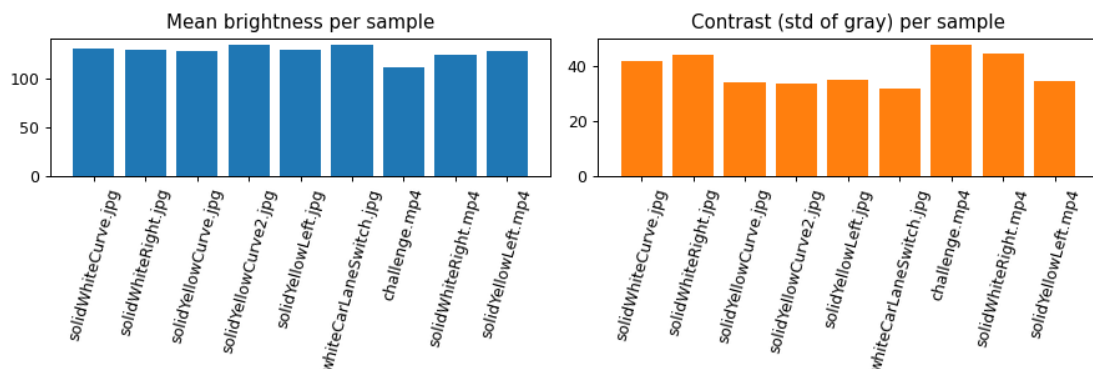


Figure 1: Brightness and contrast of each sample

4. AI Tool Used & Purpose

Honest note on tooling: the lane detector itself uses classical computer vision (OpenCV: Gaussian blur, Canny edge detector, Hough transform). These are algorithmic image-processing methods rather than learned models. The learning-based component (a deep segmentation network such as U-Net, trained on TuSimple) is planned as future work and will be compared against this baseline.

Tool	Purpose
OpenCV (Python)	Image processing pipeline and video I/O
NumPy / Pandas	Numerical work, slope averaging, result tables
Matplotlib	Figures for analysis and results
Jupyter Notebook	Reproducible walkthrough
Claude (AI assistant, Anthropic)	Assisted with code structure, debugging a metric, and drafting this report. Prompts/screenshots are included in the evidence folder.
Planned: TensorFlow/PyTorch	Deep learning lane segmentation (future work)

5. Work Completed

The pipeline processes each frame in the following steps (implemented in lane_detection.py):

- Convert to grayscale and apply a 5×5 Gaussian blur to suppress noise.
- Canny edge detection (thresholds 50 / 150).
- Mask edges with a trapezoidal region of interest covering the road ahead of the car.

- Probabilistic Hough transform to find line segments.
- Split segments into left and right by slope sign, drop near-horizontal segments ($|\text{slope}| < 0.45$), and compute a length-weighted average line for each side; extrapolate to the bottom of the image.
- Draw the lane lines and a translucent lane area on the original frame.
- For video: exponential smoothing ($\alpha = 0.2$) of each lane line across frames to reduce flicker.

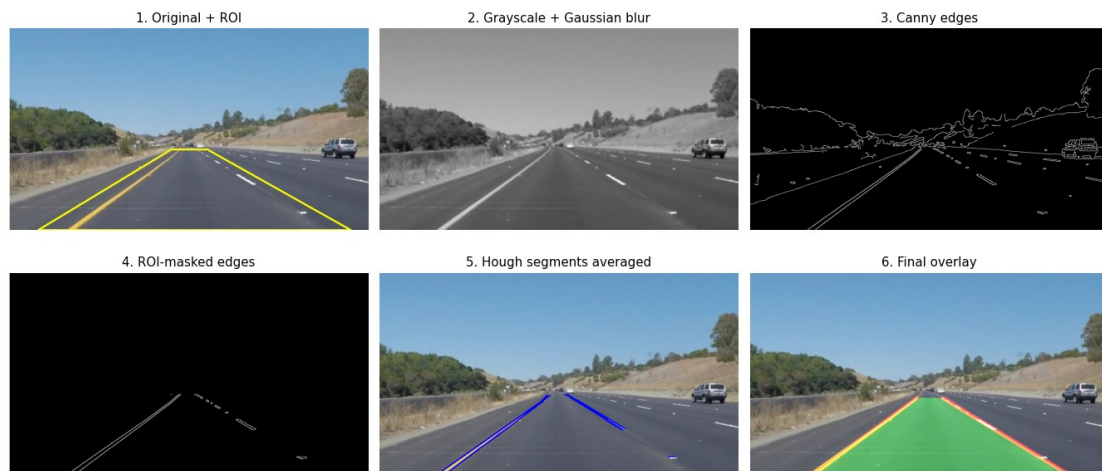


Figure 2: Pipeline stages on solidYellowCurve.jpg

6. Initial Results

6.1 Still images

Both lane lines were found on all 6 test images (6/6). Per-image processing time was about 5–8 ms (the first image, 20.6 ms, includes warm-up).



Figure 3: Detected lanes on the six sample images

6.2 Videos

Video	Mode	Frames	Both lanes found	Jitter (px/frame)	Speed (FPS)
solidWhiteRight	raw	221	100.0%	4.34	82.5
solidWhiteRight	smoothed	221	100.0%	0.90	83.7
solidYellowLeft	raw	681	100.0%	5.29	84.2

Video	Mode	Frames	Both lanes found	Jitter (px/frame)	Speed (FPS)
solidYellowLeft	smoothed	681	100.0%	0.98	81.0
challenge	raw	251	98.8%	27.67	36.5
challenge	smoothed	251	98.8%	6.60	36.9

Jitter is the mean frame-to-frame change, in pixels, of the lane position at the bottom of the image (lower is more stable). Temporal smoothing reduced jitter by roughly 4–5× on every video, with no loss in speed. The pipeline runs at roughly 80+ FPS on 960×540 video and about 36 FPS on 1280×720 on the CPU used here, fast enough for real-time use.

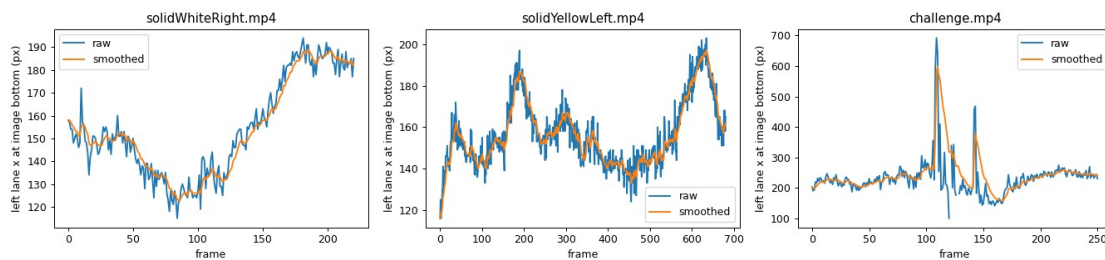


Figure 4: Left-lane position over time, raw vs smoothed

6.3 Limitations observed

"Both lanes found" means the detector produced a left and a right line; it does not prove the lines are correct, because this dataset has no ground-truth labels. Visual inspection of the challenge video shows a real failure: in some frames the left line is pulled onto the shadowed road edge rather than the yellow marking (Figure 5). The method also assumes straight lane lines, so strong curves are only approximated, and it uses a fixed region of interest.



Figure 5: Failure case – left line offset from the yellow marking (challenge.mp4, frame 90)

7. Current Status & Work Remaining

Item	Status
Classical OpenCV pipeline (images)	Done
Video processing + temporal smoothing	Done
Dataset analysis (sample set)	Done

Item	Status
Notebook, code, results, README	Done (draft)
Switch to TuSimple dataset with labels	To do
Accuracy vs ground truth (TuSimple accuracy, false positives / negatives)	To do
Curved-lane handling (perspective transform + polynomial fit)	To do
Robustness to shadows (e.g. colour-space thresholding, CLAHE)	To do
Deep learning model (U-Net) and comparison with baseline	To do
Final report and presentation	To do

Work split (to be confirmed by team): Student 1 – dataset loading and analysis; Student 2 – image-processing pipeline and tuning; Student 3 – evaluation, results, report and slides.

8. References

- Udacity Self-Driving Car Nanodegree, CarND-LaneLines-P1 – <https://github.com/udacity/CarND-LaneLines-P1>
- TuSimple Lane Detection Benchmark (Kaggle mirror) – <https://www.kaggle.com/datasets/manideep1108/tusimple>
- OpenCV documentation – <https://docs.opencv.org>